

Performance Evaluation of Baseline and Optimized TensorFlow Feedforward Neural Networks for Diabetes Prediction

William Lorenzo^{1*}

^{1*}Department of Statistics, B.S.: Data Science Concentration,
California State University, East Bay, 25800 Carlos Bee Boulevard,
Hayward, California, 94542, USA.

Corresponding author(s). E-mail(s): wlorenzo@horizon.csueastbay.edu;

Abstract

Diabetes mellitus is a chronic metabolic condition associated with elevated blood glucose and long-term health complications. Early screening can support preventive care, but predictive modeling remains challenging when medical features are noisy, incomplete, or imbalanced. This study evaluates baseline and optimized TensorFlow feedforward neural networks for binary diabetes prediction using the Pima Indians Diabetes dataset.

Keywords: TensorFlow, feedforward neural network, multilayer perceptron, diabetes prediction, Pima Indians Diabetes dataset, binary classification, class imbalance, medical machine learning

1 Introduction

Diabetes mellitus is a major public health concern because it is associated with abnormal blood glucose regulation and increased risk of cardiovascular disease, kidney disease, neuropathy, retinopathy, and other complications. Machine learning models can support diabetes screening by learning relationships between diagnostic measurements and disease outcome. In this project, TensorFlow was used to develop and compare feedforward neural network classifiers for diabetes prediction using the Pima Indians Diabetes dataset.

The Pima Indians Diabetes dataset is a widely used benchmark dataset in machine learning and medical classification. It has historically been associated with work from the National Institute of Diabetes and Digestive and Kidney Diseases and with the study by Smith et al. on forecasting the onset of diabetes mellitus using the ADAP learning algorithm [1–3]. The commonly used Kaggle version contains 768 observations and nine columns: eight diagnostic predictor variables and one binary outcome variable [2]. All records correspond to female patients at least 21 years old of Pima Indian heritage [3, 4].

The primary objective of this study is to evaluate whether an optimized TensorFlow feedforward neural network improves diabetes prediction compared with a simpler baseline network. The optimized model incorporates dropout, L2 regularization, class weighting, adaptive learning-rate reduction, and early stopping. These techniques are often used to improve generalization and stabilize neural network training [7, 9–11].

This project also emphasizes correct preprocessing. Several variables in the Pima Indians Diabetes dataset contain zero values that are not physiologically meaningful, including glucose, blood pressure, body mass index, skin thickness, and insulin. Treating these values as missing and imputing them using training-set medians helps reduce bias while avoiding test-set leakage. Because the positive diabetes class is less frequent than the non-diabetes class, class weighting was used to improve sensitivity to diabetic cases.

2 Dataset History and Source

The Pima Indians Diabetes dataset originated from diabetes research associated with the National Institute of Diabetes and Digestive and Kidney Diseases. The dataset was designed for predicting the onset of diabetes using diagnostic measurements. Smith et al. used the ADAP learning algorithm to forecast diabetes mellitus and reported this work in the 1988 Symposium on Computer Applications and Medical Care [1]. Later versions of the dataset became widely circulated through machine learning repositories and software packages, including UCI-related distributions and Kaggle-hosted copies [2–4].

The dataset used in this project was loaded from a local file named `diabetes.csv`. This file follows the common Kaggle format of the Pima Indians Diabetes Database, consisting of the following variables:

- **Pregnancies:** number of times pregnant
- **Glucose:** plasma glucose concentration
- **BloodPressure:** diastolic blood pressure
- **SkinThickness:** triceps skin fold thickness
- **Insulin:** two-hour serum insulin
- **BMI:** body mass index
- **DiabetesPedigreeFunction:** diabetes pedigree function
- **Age:** age in years
- **Outcome:** binary diabetes outcome, where 0 indicates non-diabetic and 1 indicates diabetic

The dataset contains 768 observations. The target distribution is moderately imbalanced, with approximately 500 non-diabetic cases and 268 diabetic cases. This imbalance motivates the use of class weighting during model training.

3 Exploratory Data Analysis

The dataset contained 768 rows and 9 columns. All predictor variables were numerical, and the response variable `Outcome` was binary. Initial summary statistics showed that the variables `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, and `BMI` contained zero values. Since a value of zero is not physiologically meaningful for these variables, these entries were interpreted as missing observations rather than true values.

The initial summary statistics are shown in Table 1. These values are based on the raw dataset before replacing impossible zero values with missing values.

Table 1 Summary statistics for the raw Pima Indians Diabetes dataset

Variable	Mean	Std. Dev.	Min	Max
Pregnancies	3.85	3.37	0.00	17.00
Glucose	120.89	31.97	0.00	199.00
BloodPressure	69.11	19.36	0.00	122.00
SkinThickness	20.54	15.95	0.00	99.00
Insulin	79.80	115.24	0.00	846.00
BMI	31.99	7.88	0.00	67.10
DiabetesPedigreeFunction	0.47	0.33	0.08	2.42
Age	33.24	11.76	21.00	81.00
Outcome	0.35	0.48	0.00	1.00

3.1 Exploratory figures

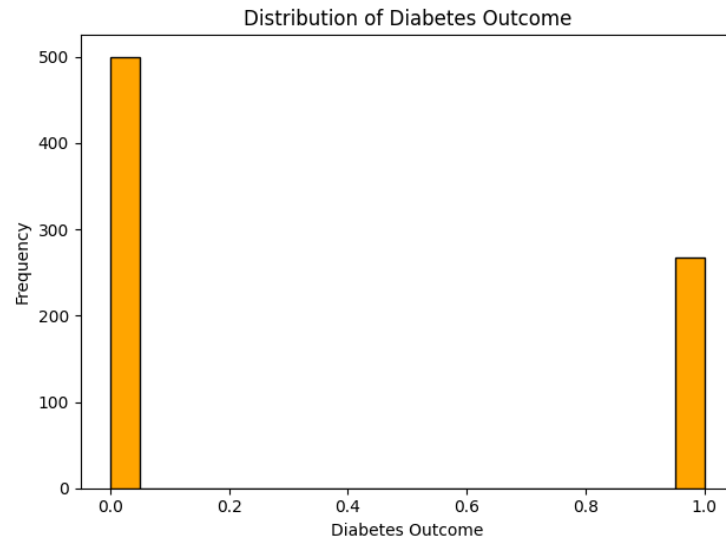


Fig. 1 Distribution of the binary diabetes outcome variable.

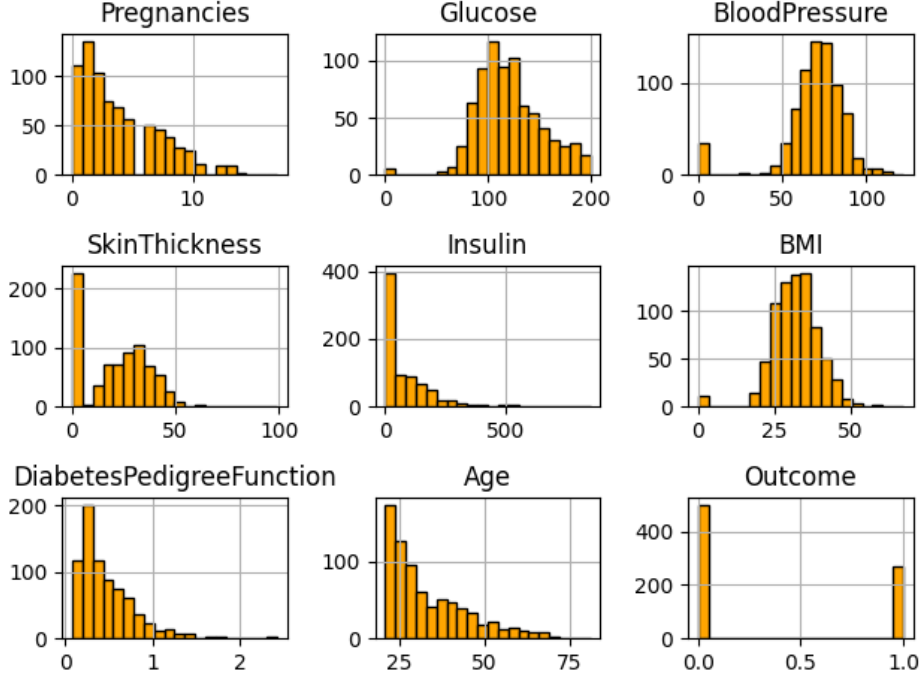


Fig. 2 Histograms of the predictor variables in the Pima Indians Diabetes dataset.

4 Methods

4.1 Preprocessing workflow

The preprocessing workflow was designed to avoid data leakage and preserve the integrity of the test set. First, impossible zero values in **Glucose**, **BloodPressure**, **SkinThickness**, **Insulin**, and **BMI** were replaced with missing values. The predictor matrix X consisted of all eight medical variables, and the response vector y was defined as **Outcome**. A stratified 70/30 training-testing split was then performed using `random_state = 452`. Stratification preserved the class distribution across training and testing subsets.

Missing values were imputed using medians computed from the training set only. These training medians were applied to both the training and testing sets. This is important because computing imputation values from the full dataset would allow information from the test set to influence preprocessing. After imputation, predictors were standardized using **StandardScaler**. The scaler was fit only on the training set and then applied to both the training and testing data.

For a feature x_j , standardization was performed as

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \quad j = 1, 2, \dots, p, \quad (1)$$

where μ_j and σ_j are the training-set mean and standard deviation for feature j .

Because the response classes were imbalanced, balanced class weights were computed from the training labels. The class weight for class c is defined as

$$w_c = \frac{n}{kn_c}, \quad (2)$$

where n is the number of training observations, k is the number of classes, and n_c is the number of observations in class c . The resulting class weights were approximately

$$w_0 = 0.7671, \quad w_1 = 1.4358. \quad (3)$$

Thus, the diabetic class received a larger training weight because it appeared less frequently.

4.2 Feedforward neural network formulation

A feedforward neural network maps an input vector $x \in \mathbb{R}^p$ to a predicted probability $\hat{y} \in [0, 1]$. For the first hidden layer, the transformation is

$$h^{(1)} = \phi \left(W^{(1)}x + b^{(1)} \right), \quad (4)$$

where $W^{(1)}$ is a weight matrix, $b^{(1)}$ is a bias vector, and ϕ is the rectified linear unit activation function. ReLU is defined as

$$\phi(a) = \max(0, a). \quad (5)$$

For binary classification, the output layer uses the sigmoid activation function:

$$\hat{y} = \sigma(a) = \frac{1}{1 + e^{-a}}. \quad (6)$$

The predicted class is obtained using a threshold of 0.5:

$$\hat{c} = \begin{cases} 1, & \hat{y} \geq 0.5, \\ 0, & \hat{y} < 0.5. \end{cases} \quad (7)$$

The binary cross-entropy loss is

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]. \quad (8)$$

For the tuned model, L2 regularization was added to discourage overly large weights:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \sum_{\ell} \left\| W^{(\ell)} \right\|_2^2, \quad (9)$$

where $\lambda = 0.0001$ in this project.

Dropout was also used in the tuned network. During training, dropout randomly deactivates hidden units. If $m_j \sim \text{Bernoulli}(1 - p)$ is a dropout mask, then a dropped hidden activation can be written as

$$\tilde{h}_j = m_j h_j. \quad (10)$$

This encourages the model to rely less heavily on individual neurons and can improve generalization [10].

4.3 Baseline TensorFlow model

The baseline model was a feedforward neural network implemented with the TensorFlow/Keras `Sequential` API. The model architecture consisted of:

- Dense hidden layer with 64 units and ReLU activation
- Dense hidden layer with 32 units and ReLU activation
- Output layer with 1 unit and sigmoid activation

The baseline model contained 2,689 trainable parameters. It was compiled using the Adam optimizer, binary cross-entropy loss, and accuracy as the primary metric. Training used a batch size of 32, 100 epochs, validation split of 0.30, and class weights.

4.4 Tuned TensorFlow model

The tuned model used the same general dense architecture but added regularization and training controls. The architecture consisted of:

- Dense layer with 64 units, ReLU activation, and L2 regularization $\lambda = 0.0001$
- Dropout layer with rate 0.15
- Dense layer with 32 units, ReLU activation, and L2 regularization $\lambda = 0.0001$
- Dropout layer with rate 0.05
- Output layer with 1 unit and sigmoid activation

The tuned model also contained 2,689 trainable parameters. It was compiled with Adam using a learning rate of 0.0003, binary cross-entropy loss, and accuracy. Two callbacks were applied during training:

- `EarlyStopping`, monitoring validation loss with patience 6 and restoring the best weights
- `ReduceLROnPlateau`, monitoring validation loss with factor 0.5, patience 3, and minimum learning rate 10^{-5}

The tuned model used the same 70/30 train-test split, standardized predictors, validation split, batch size, and class weights as the baseline model.

4.5 Modeling workflow algorithm

Algorithm 1 TensorFlow workflow for diabetes prediction

- 1: Load `diabetes.csv` into a pandas DataFrame
 - 2: Inspect dimensions, data types, summary statistics, and target distribution
 - 3: Replace biologically implausible zeros in selected medical variables with missing values
 - 4: Define predictor matrix X using eight medical predictors
 - 5: Define binary response vector y using `Outcome`
 - 6: Split data into stratified training and testing sets using a 70/30 partition
 - 7: Compute medians from the training predictors only
 - 8: Impute missing training and testing values using training-set medians
 - 9: Standardize predictors using training-set means and standard deviations
 - 10: Compute balanced class weights using training labels
 - 11: Train the baseline TensorFlow feedforward neural network
 - 12: Train the tuned TensorFlow feedforward neural network with dropout, L2 regularization, adaptive learning-rate reduction, and early stopping
 - 13: Evaluate both models on training and testing sets
 - 14: Generate loss curves, accuracy curves, confusion matrices, and classification reports
 - 15: Compare model performance using accuracy, precision, recall, and F1-score
-

5 Results

5.1 Baseline model performance

The baseline model achieved training accuracy of 0.8678 and testing accuracy of 0.7879. The training loss was 0.3419, while the testing loss was 0.5358. The increasing gap between training and validation loss during later epochs indicated overfitting: the model continued improving on the training data while validation performance became unstable.

The baseline test classification report is shown in Table 2. The model achieved a weighted F1-score of 0.78. For the diabetic class, precision was 0.72, recall was 0.64, and F1-score was 0.68. This indicates that the baseline model correctly identified many diabetic cases but still missed a meaningful proportion of them.

Table 2 Baseline model classification report on the test set

Class	Precision	Recall	F1-score	Support
0	0.82	0.87	0.84	150
1	0.72	0.64	0.68	81
Accuracy			0.79	231
Macro Avg.	0.77	0.75	0.76	231
Weighted Avg.	0.78	0.79	0.78	231

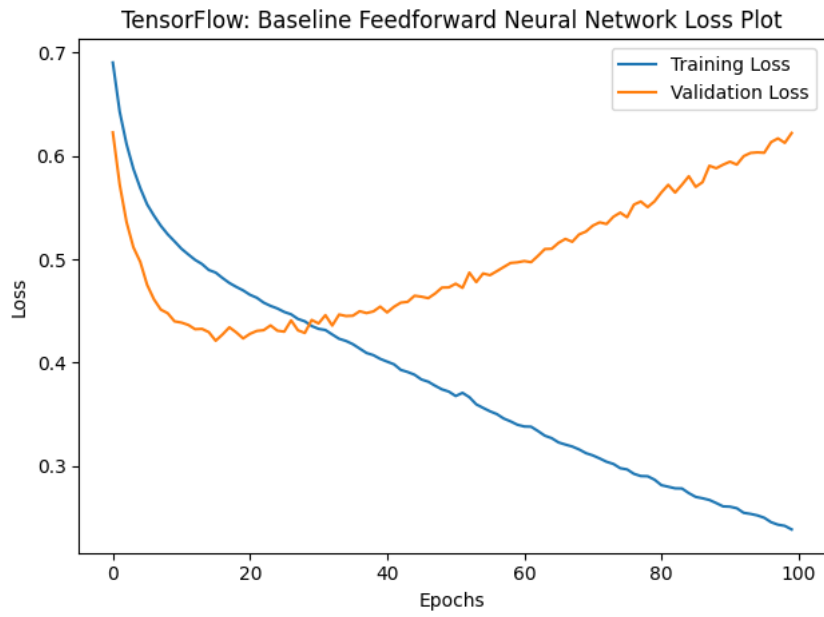


Fig. 3 Training and validation loss curves for the baseline TensorFlow feedforward neural network.

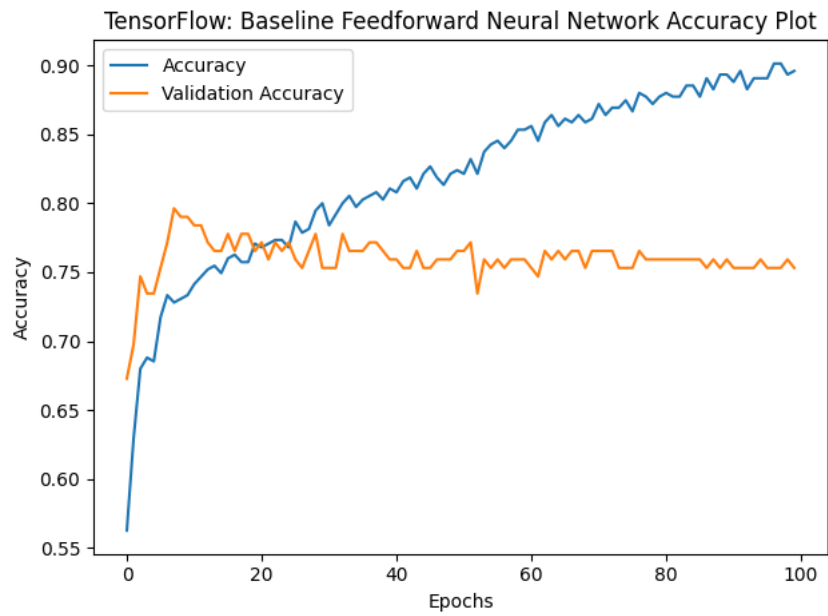


Fig. 4 Training and validation accuracy curves for the baseline TensorFlow feedforward neural network.

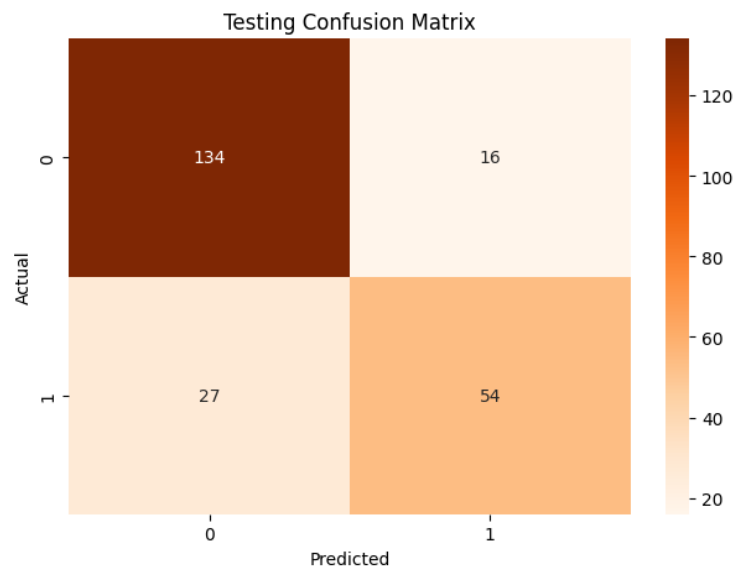


Fig. 5 Testing confusion matrix for the baseline model.

5.2 Tuned model performance

The tuned model achieved training accuracy of 0.7765 and testing accuracy of 0.8182. The training loss was 0.4812, while the testing loss was 0.4255. Unlike the baseline model, the tuned model produced a smoother validation-loss curve and stronger holdout accuracy, suggesting better generalization.

The tuned test classification report is shown in Table 3. The tuned model achieved a weighted F1-score of 0.82. The diabetic-class recall improved from 0.64 in the baseline model to 0.72 in the tuned model. This is an important improvement because false negatives are especially concerning in medical screening tasks.

Table 3 Tuned model classification report on the test set

Class	Precision	Recall	F1-score	Support
0	0.85	0.87	0.86	150
1	0.75	0.72	0.73	81
Accuracy			0.82	231
Macro Avg.	0.80	0.79	0.80	231
Weighted Avg.	0.82	0.82	0.82	231

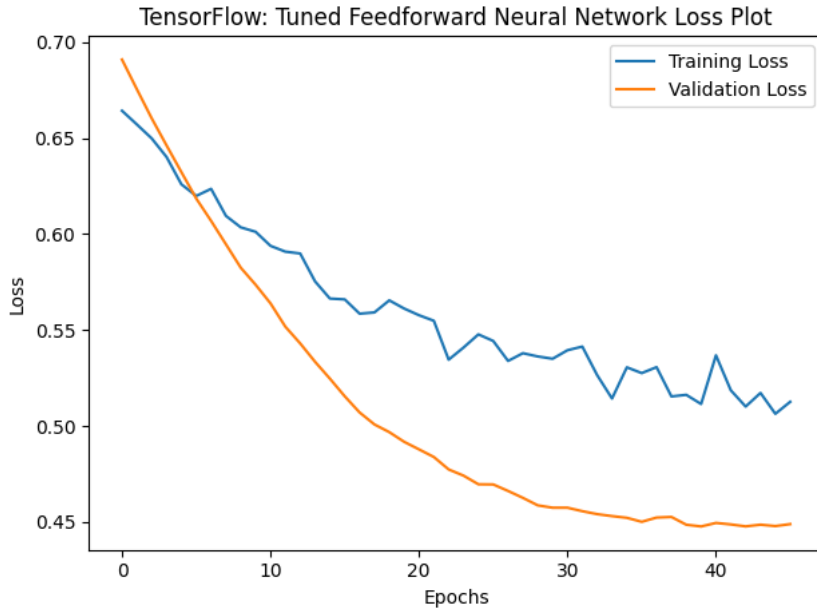


Fig. 6 Training and validation loss curves for the tuned TensorFlow feedforward neural network.

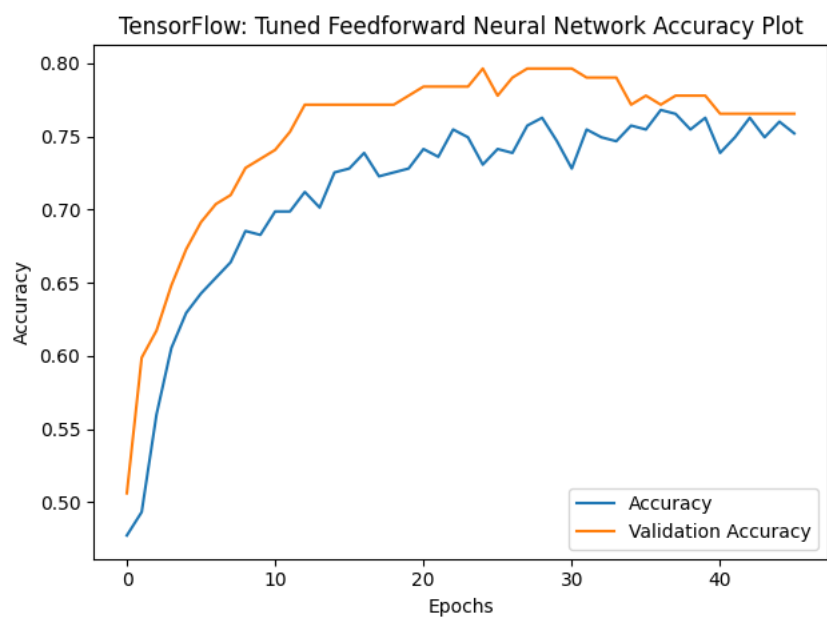


Fig. 7 Training and validation accuracy curves for the tuned TensorFlow feedforward neural network.

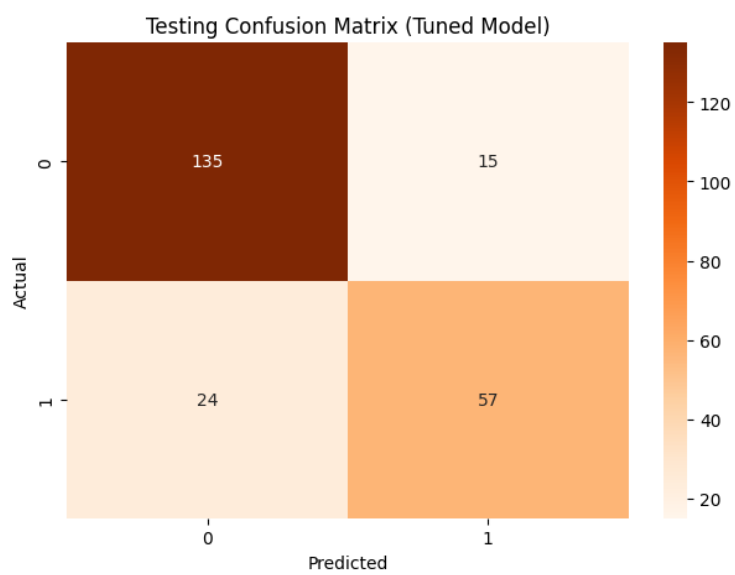


Fig. 8 Testing confusion matrix for the tuned model.

5.3 Performance comparison

Table 4 compares the baseline and tuned TensorFlow models. The tuned model improved test accuracy from 0.7879 to 0.8182 and reduced test loss from 0.5358 to 0.4255. More importantly, recall for the diabetic class improved from 0.64 to 0.72.

Table 4 Comparison of baseline and tuned TensorFlow feedforward neural networks

Metric	Baseline FNN	Tuned FNN
Train Accuracy	0.8678	0.7765
Test Accuracy	0.7879	0.8182
Train Loss	0.3419	0.4812
Test Loss	0.5358	0.4255
Class 0 Test Precision	0.82	0.85
Class 0 Test Recall	0.87	0.87
Class 1 Test Precision	0.72	0.75
Class 1 Test Recall	0.64	0.72
Class 1 Test F1-score	0.68	0.73
Weighted F1-score	0.78	0.82

5.4 Interpretation of class-level results

The class-level results indicate that the tuned model is more balanced than the baseline model. The baseline model achieved higher training accuracy but lower test accuracy, suggesting that it learned patterns specific to the training data. The tuned model had lower training accuracy but higher test accuracy, which indicates stronger generalization.

For diabetes screening, recall for class 1 is especially important because class 1 represents diabetic patients. A false negative means that a diabetic patient is predicted as non-diabetic. The tuned model improved class 1 recall from 0.64 to 0.72, meaning it identified a larger proportion of diabetic patients in the test set. Although the tuned model remains imperfect, it provides a better balance between precision and recall than the baseline model.

6 Discussion

This project demonstrates that careful preprocessing and regularized neural network training can improve diabetes prediction using the Pima Indians Diabetes dataset. The baseline feedforward neural network achieved strong training performance but showed evidence of overfitting, as its training loss decreased while validation loss increased in later epochs. The tuned model produced better test accuracy and a more balanced classification report.

A key improvement was the use of class weights. The dataset is moderately imbalanced, with fewer diabetic cases than non-diabetic cases. Without weighting, a model

may learn to favor the majority class. Class weighting increased the training penalty for misclassifying diabetic observations, which helped improve recall for class 1.

The tuned model used dropout and L2 regularization. These methods intentionally restrict model flexibility during training. As a result, the tuned model had lower training accuracy than the baseline model. However, the tuned model performed better on the test set, which indicates improved generalization. This tradeoff is desirable because the purpose of the model is not to memorize the training data but to predict unseen cases accurately.

In medical screening, accuracy alone is not sufficient. A model can achieve high accuracy while failing to identify many positive cases. For this reason, recall and F1-score for the diabetic class are especially important. The tuned model improved diabetic-class recall from 0.64 to 0.72 and improved diabetic-class F1-score from 0.68 to 0.73.

Several limitations should be noted. First, the dataset is relatively small, containing only 768 observations. Second, the analysis used a single train-test split rather than repeated cross-validation. Third, the dataset contains historical records from a specific population, so generalization to broader populations should be interpreted cautiously. Fourth, feedforward neural networks are not always the best models for small tabular datasets; future work could compare TensorFlow models with logistic regression, random forests, support vector machines, and gradient boosting methods [15–18].

7 Conclusion

This study evaluated baseline and optimized TensorFlow feedforward neural networks for diabetes prediction using the Pima Indians Diabetes dataset. The workflow included exploratory data analysis, replacement of impossible zero values, median imputation using training-set statistics, standardization, class weighting, baseline neural network training, tuned neural network training, and classification-report evaluation.

The baseline model achieved 0.7879 test accuracy and a class 1 recall of 0.64. The tuned model achieved 0.8182 test accuracy and a class 1 recall of 0.72. Therefore, the tuned model produced better overall generalization and improved detection of diabetic cases. The results suggest that class weighting, dropout, L2 regularization, adaptive learning-rate reduction, and early stopping can improve neural network performance on small, moderately imbalanced medical datasets.

Future work should include stratified k-fold cross-validation, threshold tuning to optimize diabetic-class recall, calibration analysis, receiver operating characteristic analysis, and comparison with classical machine learning methods. These additions would provide a more comprehensive assessment of model reliability for diabetes screening.

Supplementary information. Supplementary material may include the full Python/TensorFlow code, exported training-history plots, confusion matrix visualizations, and additional model-comparison outputs.

Acknowledgements. The author acknowledges California State University, East Bay for academic support and the instructional environment that supported this TensorFlow machine learning project.

Declarations

Funding

No external funding was reported for this study.

Conflict of interest/Competing interests

The author declares no competing interests.

Ethics approval and consent to participate

Not applicable. This study used a publicly available benchmark dataset and did not involve direct interaction with human participants, animal subjects, or identifiable private information.

Consent for publication

Not applicable.

Data availability

The dataset used in this study follows the commonly available Pima Indians Diabetes Database format and was loaded locally as `diabetes.csv`. Public versions are available through Kaggle and related machine learning dataset repositories [2–4].

Materials availability

Not applicable.

Code availability

The TensorFlow, pandas, NumPy, scikit-learn, Matplotlib, and seaborn code used for preprocessing, model training, and evaluation is included in Appendix D.

Author contribution

William Lorenzo contributed to data preprocessing, exploratory data analysis, TensorFlow modeling, interpretation of results, and manuscript drafting.

Appendix A Reproducibility Notes

Key implementation details are summarized below:

- Programming language: Python
- Major libraries: NumPy, pandas, Matplotlib, seaborn, scikit-learn, TensorFlow/Keras
- Dataset file: `diabetes.csv`
- Train-test split: 70/30 with `random_state = 452`
- Stratification: `stratify = y`
- Missing-value strategy: replace impossible zeros with `NaN`, then impute using training medians
- Scaling: `StandardScaler` fit on training predictors only
- Loss function: binary cross-entropy
- Output activation: sigmoid
- Baseline epochs: 100
- Tuned epochs: up to 100 with early stopping

- Class imbalance handling: balanced class weights

Appendix B Recommended Figure File Names

Upload the following figure files to Overleaf after exporting them from Python:

- tensorflow_logo-2.png
- target_distribution.png
- feature_histogram.png
- baseline_loss_curve.png
- baseline_accuracy_curve.png
- baseline_train_confusion_matrix.png
- baseline_test_confusion_matrix.png
- tuned_loss_curve.png
- tuned_accuracy_curve.png
- tuned_train_confusion_matrix.png
- tuned_test_confusion_matrix.png

Appendix C Additional Training Confusion Matrices

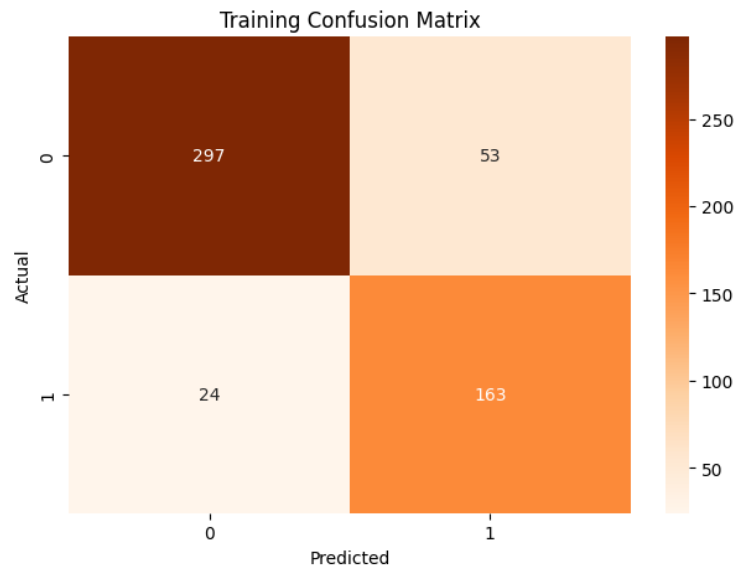


Fig. C1 Training confusion matrix for the baseline model.

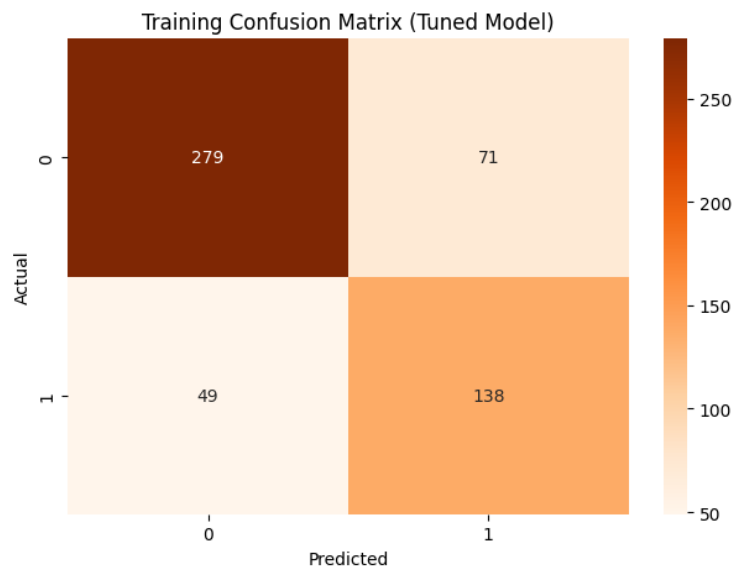


Fig. C2 Training confusion matrix for the tuned model.

Appendix D Full TensorFlow Code

Listing 1 Complete TensorFlow workflow for diabetes prediction

```

1 # Loading Libraries
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.model_selection import train_test_split
8 from sklearn.preprocessing import StandardScaler
9 from sklearn.metrics import accuracy_score, confusion_matrix,
   classification_report
10 from sklearn.utils.class_weight import compute_class_weight
11
12 import tensorflow as tf
13 from tensorflow.keras.models import Sequential
14 from tensorflow.keras.layers import Input, Dense, Dropout
15 from tensorflow.keras.optimizers import Adam
16 from tensorflow.keras.regularizers import l2
17 from tensorflow.keras.callbacks import EarlyStopping,
   ReduceLROnPlateau
18
19 # Loading Dataset
20 df = pd.read_csv("diabetes.csv")
21 print(df.head())

```

```

22
23 # Summary Statistics
24 print(df.describe())
25
26 # Dataset Dimension
27 print(df.shape)
28
29 # Data Types
30 print(df.info())
31
32 # Target Distribution
33 plt.figure()
34 plt.hist(df["Outcome"], bins=20, color="orange", edgecolor="black
35 ")
36 plt.xlabel("Diabetes Outcome")
37 plt.ylabel("Frequency")
38 plt.title("Diabetes Outcome Distribution")
39 plt.savefig("target_distribution.png", dpi=300, bbox_inches="
40 tight")
41 plt.show()
42
43 # Feature Histograms
44 plt.figure()
45 df.hist(color="orange", edgecolor="black", bins=20)
46 plt.savefig("feature_histograms.png", dpi=300, bbox_inches="tight
47 ")
48 plt.show()
49
50 # Replacing impossible 0s with missing values
51 cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "
52 BMI"]
53 df[cols] = df[cols].replace(0, np.nan)
54
55 # Predictors
56 X = df[[
57     "Pregnancies",
58     "Glucose",
59     "BloodPressure",
60     "SkinThickness",
61     "Insulin",
62     "BMI",
63     "DiabetesPedigreeFunction",
64     "Age"
65 ]]
66
67 # Response Variable
68 y = df["Outcome"]
69
70 # Data Splitting
71 X_train, X_test, y_train, y_test = train_test_split(

```

```

68     X,
69     y,
70     test_size=0.3,
71     random_state=452,
72     stratify=y
73 )
74
75 # Checking missing values after replacement
76 na_counter = df.isna().sum().sum()
77 print(f"NA: {na_counter}")
78
79 # Filling Missing Values using TRAIN medians
80 train_medians = X_train.median()
81 X_train = X_train.fillna(train_medians)
82 X_test = X_test.fillna(train_medians)
83
84 print("Missing values in X_train:", X_train.isna().sum().sum())
85 print("Missing values in X_test:", X_test.isna().sum().sum())
86
87 # Class Weights
88 classes = np.unique(y_train)
89 weights = compute_class_weight(
90     class_weight="balanced",
91     classes=classes,
92     y=y_train
93 )
94 class_weights = dict(zip(classes, weights))
95 print(class_weights)
96
97 # Scaling
98 scaler = StandardScaler()
99 X_train_scaled = scaler.fit_transform(X_train)
100 X_test_scaled = scaler.transform(X_test)
101
102 # -----
103 # Baseline Sequential Architecture
104 # -----
105
106 tf1 = Sequential([
107     Input(shape=(X_train_scaled.shape[1],)),
108     Dense(64, activation="relu"),
109     Dense(32, activation="relu"),
110     Dense(1, activation="sigmoid")
111 ])
112
113 tf1.summary()
114
115 # Compile baseline model
116 tf1.compile(
117     optimizer="adam",

```

```

118     loss="binary_crossentropy",
119     metrics=["accuracy"]
120 )
121
122 # Fit baseline model
123 tf1_fit = tf1.fit(
124     X_train_scaled,
125     y_train,
126     validation_split=0.3,
127     class_weight=class_weights,
128     batch_size=32,
129     epochs=100,
130     verbose=1
131 )
132
133 # Evaluation on Train Set
134 loss, accuracy = tf1.evaluate(X_train_scaled, y_train)
135 print("Baseline Train Loss:", loss)
136 print("Baseline Train Accuracy:", accuracy)
137
138 # Evaluation on Test Set
139 loss, accuracy = tf1.evaluate(X_test_scaled, y_test)
140 print("Baseline Test Loss:", loss)
141 print("Baseline Test Accuracy:", accuracy)
142
143 # Baseline Loss Plot
144 plt.figure()
145 plt.plot(tf1_fit.history["loss"], label="Training Loss")
146 plt.plot(tf1_fit.history["val_loss"], label="Validation Loss")
147 plt.legend()
148 plt.title("TensorFlow: Baseline Feedforward Neural Network Loss
149         Plot")
150 plt.xlabel("Epochs")
151 plt.ylabel("Loss")
152 plt.savefig("baseline_loss_curve.png", dpi=300, bbox_inches="
153         tight")
154 plt.show()
155
156 # Baseline Accuracy Plot
157 plt.figure()
158 plt.plot(tf1_fit.history["accuracy"], label="Accuracy")
159 plt.plot(tf1_fit.history["val_accuracy"], label="Validation
160         Accuracy")
161 plt.legend()
162 plt.title("TensorFlow: Baseline Feedforward Neural Network
163         Accuracy Plot")
164 plt.xlabel("Epochs")
165 plt.ylabel("Accuracy")
166 plt.savefig("baseline_accuracy_curve.png", dpi=300, bbox_inches="
167         tight")

```

```

163 plt.show()
164
165 # Baseline Training Predictions
166 tf1_train_probs = tf1.predict(X_train_scaled)
167 tf1_train_pred = (tf1_train_probs > 0.5).astype(int)
168
169 # Baseline Training Confusion Matrix
170 tf1_cm_train = confusion_matrix(y_train, tf1_train_pred)
171 plt.figure()
172 sns.heatmap(tf1_cm_train, annot=True, fmt="d", cmap="Oranges")
173 plt.title("Training Confusion Matrix: Baseline Model")
174 plt.xlabel("Predicted")
175 plt.ylabel("Actual")
176 plt.savefig("baseline_train_confusion_matrix.png", dpi=300,
177             bbox_inches="tight")
178 plt.show()
179
180 # Baseline Testing Predictions
181 tf1_test_probs = tf1.predict(X_test_scaled)
182 tf1_test_pred = (tf1_test_probs > 0.5).astype(int)
183
184 # Baseline Testing Confusion Matrix
185 tf1_cm_test = confusion_matrix(y_test, tf1_test_pred)
186 plt.figure()
187 sns.heatmap(tf1_cm_test, annot=True, fmt="d", cmap="Oranges")
188 plt.title("Testing Confusion Matrix: Baseline Model")
189 plt.xlabel("Predicted")
190 plt.ylabel("Actual")
191 plt.savefig("baseline_test_confusion_matrix.png", dpi=300,
192             bbox_inches="tight")
193 plt.show()
194
195 # Baseline Classification Reports
196 print("BASELINE MODEL TRAIN REPORT")
197 print(classification_report(y_train, tf1_train_pred))
198
199 print("BASELINE MODEL TEST REPORT")
200 print(classification_report(y_test, tf1_test_pred))
201
202 # -----
203 # Tuned Sequential Architecture
204 # -----
205
206 tf2 = Sequential([
207     Input(shape=(X_train_scaled.shape[1],)),
208     Dense(64, activation="relu", kernel_regularizer=l2(0.0001)),
209     Dropout(0.15),
210     Dense(32, activation="relu", kernel_regularizer=l2(0.0001)),

```

```

211         Dropout(0.05),
212
213         Dense(1, activation="sigmoid")
214     ])
215
216     tf2.summary()
217
218     # Compile tuned model
219     tf2.compile(
220         optimizer=Adam(0.0003),
221         loss="binary_crossentropy",
222         metrics=["accuracy"]
223     )
224
225     # Early Stopping
226     early_stop = EarlyStopping(
227         monitor="val_loss",
228         patience=6,
229         restore_best_weights=True
230     )
231
232     # Learning Rate Scheduler
233     reduce_lr = ReduceLROnPlateau(
234         monitor="val_loss",
235         factor=0.5,
236         patience=3,
237         min_lr=1e-5
238     )
239
240     # Fit tuned model
241     tf2_fit = tf2.fit(
242         X_train_scaled,
243         y_train,
244         validation_split=0.3,
245         class_weight=class_weights,
246         batch_size=32,
247         epochs=100,
248         callbacks=[reduce_lr, early_stop],
249         verbose=1
250     )
251
252     # Evaluation on Train Set
253     loss, accuracy = tf2.evaluate(X_train_scaled, y_train)
254     print("Tuned Train Loss:", loss)
255     print("Tuned Train Accuracy:", accuracy)
256
257     # Evaluation on Test Set
258     loss, accuracy = tf2.evaluate(X_test_scaled, y_test)
259     print("Tuned Test Loss:", loss)
260     print("Tuned Test Accuracy:", accuracy)

```

```

261
262 # Tuned Loss Plot
263 plt.figure()
264 plt.plot(tf2_fit.history["loss"], label="Training Loss")
265 plt.plot(tf2_fit.history["val_loss"], label="Validation Loss")
266 plt.legend()
267 plt.title("TensorFlow: Tuned Feedforward Neural Network Loss Plot
268 ")
269 plt.xlabel("Epochs")
270 plt.ylabel("Loss")
271 plt.savefig("tuned_loss_curve.png", dpi=300, bbox_inches="tight")
272 plt.show()
273
274 # Tuned Accuracy Plot
275 plt.figure()
276 plt.plot(tf2_fit.history["accuracy"], label="Accuracy")
277 plt.plot(tf2_fit.history["val_accuracy"], label="Validation
278 Accuracy")
279 plt.legend()
280 plt.xlabel("Epochs")
281 plt.ylabel("Accuracy")
282 plt.title("TensorFlow: Tuned Feedforward Neural Network Accuracy
283 Plot")
284 plt.savefig("tuned_accuracy_curve.png", dpi=300, bbox_inches="
285 tight")
286 plt.show()
287
288 # Tuned Training Predictions
289 tf2_train_probs = tf2.predict(X_train_scaled)
290 tf2_train_pred = (tf2_train_probs > 0.5).astype(int)
291
292 # Tuned Training Confusion Matrix
293 tf2_cm_train = confusion_matrix(y_train, tf2_train_pred)
294 plt.figure()
295 sns.heatmap(tf2_cm_train, annot=True, fmt="d", cmap="Oranges")
296 plt.title("Training Confusion Matrix: Tuned Model")
297 plt.xlabel("Predicted")
298 plt.ylabel("Actual")
299 plt.savefig("tuned_train_confusion_matrix.png", dpi=300,
300 bbox_inches="tight")
301 plt.show()
302
303 # Tuned Testing Predictions
304 tf2_test_probs = tf2.predict(X_test_scaled)
305 tf2_test_pred = (tf2_test_probs > 0.5).astype(int)
306
307 # Tuned Testing Confusion Matrix
308 tf2_cm_test = confusion_matrix(y_test, tf2_test_pred)
309 plt.figure()
310 sns.heatmap(tf2_cm_test, annot=True, fmt="d", cmap="Oranges")

```

```

306 plt.title("Testing Confusion Matrix: Tuned Model")
307 plt.xlabel("Predicted")
308 plt.ylabel("Actual")
309 plt.savefig("tuned_test_confusion_matrix.png", dpi=300,
            bbox_inches="tight")
310 plt.show()
311
312 # Tuned Classification Reports
313 print("TUNED MODEL TRAIN REPORT")
314 print(classification_report(y_train, tf2_train_pred))
315
316 print("TUNED MODEL TEST REPORT")
317 print(classification_report(y_test, tf2_test_pred))

```

References

- [1] Smith JW, Everhart JE, Dickson WC, Knowler WC, Johannes RS. Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In: *Proceedings of the Symposium on Computer Applications and Medical Care*; 1988. p. 261–265.
- [2] Kaggle. Pima Indians Diabetes Database. Available at: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>
- [3] Leisch F, Dimitriadou E. Pima Indians Diabetes Database, `mlbench` R package documentation. Available at: <https://search.r-project.org/CRAN/refmans/mlbench/html/PimaIndiansDiabetes.html>
- [4] Faraway J. Pima Diabetes Survey Dataset, R package documentation. Available at: <https://rdr.io/cran/faraway/man/pima.html>
- [5] Abadi M, Agarwal A, Barham P, et al. TensorFlow: large-scale machine learning on heterogeneous systems. 2015. Available at: <https://www.tensorflow.org/>
- [6] Chollet F, et al. Keras. 2015. Available at: <https://keras.io/>
- [7] Goodfellow I, Bengio Y, Courville A. *Deep Learning*. Cambridge, MA: MIT Press; 2016.
- [8] Haykin S. *Neural Networks and Learning Machines*. 3rd ed. Upper Saddle River, NJ: Pearson; 2009.
- [9] Kingma DP, Ba J. Adam: a method for stochastic optimization. In: *Proceedings of the 3rd International Conference on Learning Representations*; 2015.
- [10] Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine*

- Learning Research*. 2014;15:1929–1958.
- [11] Prechelt L. Early stopping — but when? In: Orr GB, Müller KR, editors. *Neural Networks: Tricks of the Trade*. Berlin: Springer; 1998. p. 55–69.
 - [12] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*. 2011;12:2825–2830.
 - [13] Japkowicz N, Stephen S. The class imbalance problem: a systematic study. *Intelligent Data Analysis*. 2002;6(5):429–449.
 - [14] He H, Garcia EA. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*. 2009;21(9):1263–1284.
 - [15] Breiman L. Random forests. *Machine Learning*. 2001;45(1):5–32.
 - [16] Cortes C, Vapnik V. Support-vector networks. *Machine Learning*. 1995;20(3):273–297.
 - [17] Friedman JH. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*. 2001;29(5):1189–1232.
 - [18] Hastie T, Tibshirani R, Friedman J. *The Elements of Statistical Learning*. 2nd ed. New York: Springer; 2009.